



C++ PROGRAMMING

(335)

STATE – 2020

Production Portion:

Program 1: Player Recruitment Reporter _____ (300 points)

TOTAL POINTS _____ (*300 points*)

Failure to adhere to any of the following rules will result in disqualification:

1. Contestant must hand in this test booklet and all printouts. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests or facsimile (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

No more than ten (10) minutes orientation

No more than 90 minutes testing time

No more than ten (10) minutes wrap-up

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.

Player Recruitment Reporter

When it comes to sports, a very select group get to play in the best of teams. To screen candidates, it is common to use a simple application to verify that the player meets certain criteria to be eligible for a spot on the team. In this exercise, you will create a C++ console application that loads in a file of player stats and generate a list of suitable candidates and for what sport they are eligible for.

Requirements:

1. You must create a C++ or C# console application named CPP_335_ContestantNumber, where ContestantNumber is your BPA assigned contestant number (including dashes). For example, CPP_335_01_2345_6789.
2. Your contestant number must appear as a comment at the top of the main source code file.
3. You must create a “Player” class that will act as the parent class for each type of sport.
 - a. The Player class shall contain the player’s name, height, weight, and GPA
 - b. An empty constructor must be created for the player class, as well as a constructor accepting an ifstream object to read the data for that player
 - c. An isCandidate function that returns a bool to indicate if the player can be a candidate
 - i. Any player must have a GPA of 2.2 in order to be eligible – this must be handled via the base class implementation of the isCandidate function
4. Child classes, inheriting from a “Player” base class must be created for the following:
 - a. “VolleyballPlayer” class with data detailing aces and kills
 - i. In order to be eligible, a player must pass the base “Player” isCandidate check as well as have at least 5 aces and 8 kills per game
 - b. “HockeyPlayer” class with data detailing goals, penalty, and plus/minus score
 - i. In order to be eligible, a player must pass the base “Player” isCandidate check as well as have at least 15 goals per season, have less than 12 penalty minutes per season, and have a plus/minus score greater than 18
 - c. “FootballPlayer” class with data detailing how long to run a 40 yard dash
 - i. In order to be eligible, a player must pass the base “Player” isCandidate check as well as have at least a 4.7 second 40 yard dash time
5. Each child class shall implement a isCandidate function that calls its parent’s isCandidate function and also enforces the above requirements for a particular sport via polymorphism.
6. Each child class shall also implement a constructor accepting an ifstream object to read the data for that player type, while also calling the base implementation of that constructor.
7. The application shall load a “players.txt” file on start and load in each player, reporting the player’s name and their sport if they are a valid candidate.
 - a. When complete, the application shall pause before exiting.
 - b. If a file is unable to be opened, an error message shall be displayed.

File Format:

```
{Sport Indicator} {Name} {Height} {Weight} {GPA} {Sport Specific Data}  
{Sport Indicator} {Name} {Height} {Weight} {GPA} {Sport Specific Data}  
{Sport Indicator} {Name} {Height} {Weight} {GPA} {Sport Specific Data}  
...
```

Here is a sample for each sport.

```
F {Name} {Height} {Weight} {GPA} {Dash Time}  
H {Name} {Height} {Weight} {GPA} {Goals} {Penalty Minutes} {Plus/Minus Score}  
V {Name} {Height} {Weight} {GPA} {Aces} {Kills}
```

Expected Output:

For the provided players.txt file, the output should be as follows:

```
Jack      : Football Candidate  
Jane      : Volleyball Candidate  
Skater    : Hockey Candidate  
Jessie    : Hockey Candidate  
Press any key to continue . . .
```

You will have 90 minutes to complete your work.

Your name and/or school name should NOT appear on any work you submit for grading.

Copy your entire solution/project to the flash drive provided. You must submit your entire solution/project so that the graders may open your project to review the source code. You must ensure that the files required to run your program are present and will execute on the flash drive provided. Note that the flash drive letter may not be the same when the program is graded as it was when you created the program. It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter. The graders will **not** compile or alter your source code to correct for this. **Submissions that do not contain source code will not be graded.**

Development Standards

- Standard name prefixes must be utilized for variables.
- All subroutines, functions, and methods must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive _____ 10 pts
The project is named according to the naming conventions _____ 10 pts

Program Execution

Code copied to USB drive and the program runs from USB _____ 10 pts

If the program does not execute, then the remaining items in this section receive a score of zero.

Application displays error message if cannot load file _____ 25 pts
Application successfully loads list of candidates _____ 30 pts
Application displays list of suitable candidates and their respective sport _____ 30 pts
Application pauses once execution is complete _____ 10 pts

Source Code Review

Code is commented at the top, for each function, and as needed _____ 15 pts
Code uses reasonable and consistent variable naming conventions _____ 10 pts
A class “Player” is implemented with suitable functions and variables _____ 15 pts
A class “VolleyballPlayer” inherits from “Player” and is implemented to spec _____ 25 pts
A class “FootballPlayer” inherits from “Player” and is implemented to spec _____ 25 pts
A class “HockeyPlayer” inherits from “Player” and is implemented to spec _____ 25 pts
Polymorphism is used properly in constructor and isCandidate functions _____ 20 pts
The program opens the input file _____ 10 pts
The program reads the input file _____ 10 pts
The program verifies that the input file was opened successfully _____ 10 pts
The program closes the input file _____ 10 pts

Total Points: 300 pts

SAMPLE SOURCE CODE:

```
main.cpp
#include <iostream>
#include <fstream> // Allows use of files
#include "FootballPlayer.h" // Allows use of FootballPlayer class
#include "HockeyPlayer.h" // Allows use of HockeyPlayer class
#include "VolleyballPlayer.h" // Allows use of VolleyballPlayer class
using namespace std;

const string PlayersFilePath = "players.txt"; // Defines dataFile path

int main()
{
    cout << left; // Sets output alignment to the left
    ifstream playerFile(PlayersFilePath); // Opens player dataFile
    if (playerFile.is_open()) // Ensures file is open
        while (!playerFile.eof()) // While not at end of file
        {
            char gameCode; // Retrieves the player's sport code
            playerFile >> gameCode;
            // Constructs the appropriate Player object,
            // Then displays a report
            if (gameCode == 'F')
                FootballPlayer(playerFile).Report("Football");
            if (gameCode == 'V')
                VolleyballPlayer(playerFile).Report("Volleyball");
            if (gameCode == 'H')
                HockeyPlayer(playerFile).Report("Hockey");
        }
    else // Reports an error if the file did not open
        cout << "Error! Problem opening file!\n";

    system("pause");
    return 0;
}
```

```
Player.cpp
#include <iostream>
#include <fstream> // Allows use of files
#include <string> // Allows use of strings
#include <iomanip> // Allows output manipulation
#include "Player.h" // Allows access to Player class
using namespace std;

// Empty constructor
Player::Player()
{
}

// Constructs via dataFile
Player::Player(ifstream &dataFile)
{
    dataFile >> this->name >> this->height
        >> this->weight >> this->gpa;
}

// Checks the base condition for candidacy
```

C++ PROGRAMMING - STATE 2020

Page 6 of 9

```
bool Player::isCandidate()
{
    return (gpa > 2.2);
}
```

```
// Retrieves the player's name
string Player::getName()
{
    return this->name;
}
```

```
// Displays a report of the player
// I elected to have the sport's name passed,
// as if I would've included it as a separate
// polymorphic method the code would have been
// elongated dramatically with virtually no
// extended utility
void Player::Report(string sport)
{
    // Invokes polymorphic isCandidate() method
    if (isCandidate())
        cout << setw(12) << getName()
            << " : " << sport << " Candidate\n";
}
```

Player.h

```
#ifndef Player_h
#define Player_h
```

```
#include <iostream>
using namespace std;
```

```
class Player
{
protected:
    string name; // Player's name
    int height; // Player's height
    int weight; // Player's weight
    double gpa; // Player's gpa
public:
    Player(); // Empty constructor
    Player(ifstream &dataFile); // Constructs via dataFile

    virtual bool isCandidate(); // Checks base conditions for candidates
    void Report(string sport); // Displays a report

    string getName(); // Retrieves the name of a player
};
```

```
#endif
```

VolleyballPlayer.cpp

```
#include <iostream>
#include <fstream> // Allows use of files
#include "VolleyballPlayer.h" // Allows use of VolleyballPlayer class
using namespace std;
```

```
// Constructs the object using a dataFile,
// Allowing the Player's constructor to
```

C++ PROGRAMMING - STATE 2020

Page 7 of 9

// initially parse the object

```
VolleyballPlayer::VolleyballPlayer(ifstream &dataFile)
    : Player(dataFile)
{
    dataFile >> this->aces >> this->kills;
}
```

// Checks if the player is a candidate

```
bool VolleyballPlayer::isCandidate()
{
    // Invokes the base Player's definition of isCandidate()
    // If any are false, return false
    if (!Player::isCandidate())
        return false;
    if (this->height < 70)
        return false;
    if (this->aces < 5)
        return false;
    if (this->kills < 8)
        return false;
    // No falsities, return true
    return true;
}
```

VolleyballPlayer.h

```
#ifndef VolleyballPlayer_h
```

```
#define VolleyballPlayer_h
```

```
#include <iostream>
```

```
#include "Player.h" // Allows access to Player class
```

```
using namespace std;
```

// Defines a Player of Volleyball

```
class VolleyballPlayer : public Player
{
private:
    int aces; // Player's aces
    int kills; // Player's kills
public:
    VolleyballPlayer(ifstream &dataFile); // Constructs via dataFile

    bool isCandidate(); // checks player's candidacy
};

#endif
```

HockeyPlayer.cpp

```
#include <iostream>
```

```
#include <fstream> // Allows file IO
```

```
#include "HockeyPlayer.h" // Allows access to HockeyPlayer class
```

```
using namespace std;
```

// Constructs the object using a dataFile,

// Allowing the Player's constructor to

// initially parse the object

```
HockeyPlayer::HockeyPlayer(ifstream &dataFile)
    : Player(dataFile)
```

```
{
    dataFile >> this->goals >> this->penaltyMinutes
        >> this->plusMinus;
}

// Checks if the player is a candidate
bool HockeyPlayer::isCandidate()
{
    // Invokes the base Player's definition of isCandidate()
    // If any are false, return false
    if (!Player::isCandidate())
        return false;
    if (this->goals < 15)
        return false;
    if (this->penaltyMinutes >= 12)
        return false;
    if (this->plusMinus < 18)
        return false;
    // No falsities, return true
    return true;
}
```

HockeyPlayer.h

```
#ifndef HockeyPlayer_h
#define HockeyPlayer_h

#include <iostream>
#include "Player.h" // Allows access to Player class
using namespace std;

// Defines a Player of Hockey
class HockeyPlayer : public Player
{
private:
    int goals; // Player's goal count
    int penaltyMinutes; // Player's time in penalty
    int plusMinus; // Player's +- score
public:
    HockeyPlayer(ifstream &dataFile); // Constructs object via dataFile

    bool isCandidate(); // Checks if player is a candidate
};

#endif
```

FootballPlayer.cpp

```
#include <iostream>
#include <fstream> // Allows file IO
#include "FootballPlayer.h" // Allows access to FootballPlayer class
using namespace std;

// Constructs the object using a datafile,
// allowing the Player constructor to
// initially parse the object
FootballPlayer::FootballPlayer(ifstream &dataFile)
```

C++ PROGRAMMING - STATE 2020

Page 9 of 9

```
        : Player(dataFile)
    {
        dataFile >> this->dashTime;
    }

// Checks if the player is a candidate
bool FootballPlayer::isCandidate()
{
    // Invokes the base Player's definition of isCandidate()
    // If any are false, return false
    if (!Player::isCandidate())
        return false;
    if (this->height <= 73)
        return false;
    if (this->weight <= 190)
        return false;
    if (this->dashTime >= 4.7)
        return false;
    // No falsities, return true
    return true;
}
```

```
FootballPlayer.h
#ifndef FootballPlayer_h
#define FootballPlayer_h

#include <iostream>
#include "Player.h" // Allows access to Player class
using namespace std;

// Defines a Player of Football
class FootballPlayer : public Player
{
private:
    double dashTime; // Player's dash time
public:
    FootballPlayer(ifstream &dataFile); // dataFile constructor

    bool isCandidate(); // Checks if the player is a candidate
};

#endif
```